

Bottom-Up Parsing

- A **bottom-up parser** creates the parse tree of the given input starting from leaves towards the root.
- A bottom-up parser tries to find the right-most derivation of the given input in the reverse order.

$S \Rightarrow \dots \Rightarrow \omega$ (the right-most derivation of ω)

$\leftarrow$ (the bottom-up parser finds the right-most derivation in the reverse order)

- Bottom-up parsing is also known as **shift-reduce parsing** because its two main actions are shift and reduce.
 - At each shift action, the current symbol in the input string is pushed to a stack.
 - At each reduction step, the symbols at the top of the stack (this symbol sequence is the right side of a production) will be replaced by the non-terminal at the left side of that production.
 - There are also two more actions: accept and error.

Shift-Reduce Parsing

- A shift-reduce parser tries to reduce the given input string into the starting symbol.

a string $\rightarrow$ the starting symbol
reduced to

- At each reduction step, a substring of the input matching to the right side of a production rule is replaced by the non-terminal at the left side of that production rule.
- If the substring is chosen correctly, the right most derivation of that string is created in the reverse order.

Rightmost Derivation: $S \xrightarrow{*}_{rm} \omega$

Shift-Reduce Parser finds: $\omega \xleftarrow{rm} \dots \xleftarrow{rm} S$

Shift-Reduce Parsing -- Example

$S \rightarrow aABb$

input string: aaabb

$A \rightarrow aA \mid a$

aaAbb

$B \rightarrow bB \mid b$

aAbb

↓ reduction

aABb

S

$S \xRightarrow{rm} aABb \xRightarrow{rm} aAbb \xRightarrow{rm} aaAbb \xRightarrow{rm} aaabb$

Right Sentential Forms

- How do we know which substring to be replaced at each reduction step?

Handle

- Informally, a **handle** of a string is a substring that matches the right side of a production rule.
 - But not every substring matches the right side of a production rule is handle
- A **handle** of a right sentential form $\gamma (\equiv \alpha\beta\omega)$ is a production rule $A \rightarrow \beta$ and a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ .

$$S \xRightarrow[\text{rm}]{*} \alpha A \omega \xRightarrow{\text{rm}} \alpha \beta \omega$$

- If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.
- We will see that ω is a string of terminals.

Handle Pruning

- A right-most derivation in reverse can be obtained by **handle-pruning**.

$$S = \gamma_0 \xRightarrow{\text{rm}} \gamma_1 \xRightarrow{\text{rm}} \gamma_2 \xRightarrow{\text{rm}} \dots \xRightarrow{\text{rm}} \gamma_{n-1} \xRightarrow{\text{rm}} \gamma_n = \omega$$

input string

- Start from γ_n , find a handle $A_n \rightarrow \beta_n$ in γ_n , and replace β_n in by A_n to get γ_{n-1} .
- Then find a handle $A_{n-1} \rightarrow \beta_{n-1}$ in γ_{n-1} , and replace β_{n-1} in by A_{n-1} to get γ_{n-2} .
- Repeat this, until we reach S .

A Shift-Reduce Parser

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Right-Most Derivation of $\text{id}+\text{id}*\text{id}$

$E \Rightarrow E+T \Rightarrow E+T * F \Rightarrow E+T * \text{id} \Rightarrow E+F * \text{id}$

$\Rightarrow E+\text{id} * \text{id} \Rightarrow T+\text{id} * \text{id} \Rightarrow F+\text{id} * \text{id} \Rightarrow \text{id}+\text{id} * \text{id}$

Right-Most Sentential Form

id + id * id

F + id * id

T + id * id

E + id * id

E + F * id

E + T * id

E + T * F

E + T

E

Reducing Production

$F \rightarrow \text{id}$

$T \rightarrow F$

$E \rightarrow T$

$F \rightarrow \text{id}$

$T \rightarrow F$

$F \rightarrow \text{id}$

$T \rightarrow T * F$

$E \rightarrow E + T$

Handles are red and underlined in the right-sentential forms.

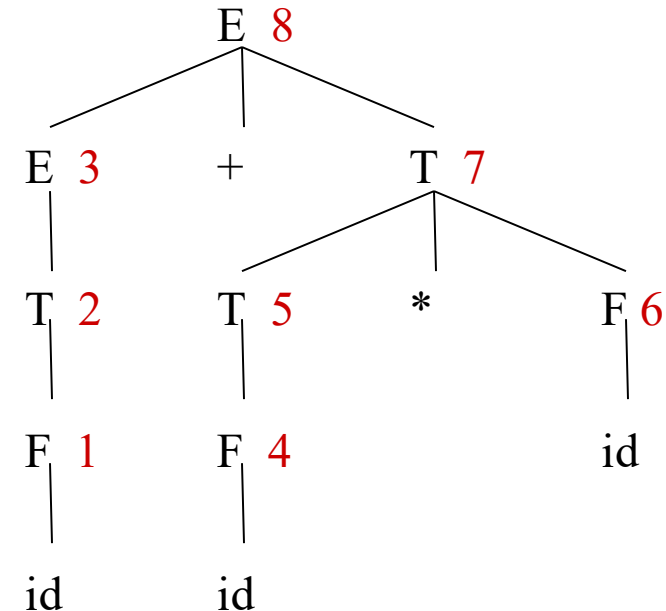
A Stack Implementation of A Shift-Reduce Parser

- There are four possible actions of a shift-parser action:
 1. **Shift** : The next input symbol is shifted onto the top of the stack.
 2. **Reduce**: Replace the handle on the top of the stack by the non-terminal.
 3. **Accept**: Successful completion of parsing.
 4. **Error**: Parser discovers a syntax error, and calls an error recovery routine.
- Initial stack just contains only the end-marker \$.
- The end of the input string is marked by the end-marker \$.

A Stack Implementation of A Shift-Reduce Parser

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	id+id*id\$	shift
\$ id	+id*id\$	reduce by $F \rightarrow id$
\$ F	+id*id\$	reduce by $T \rightarrow F$
\$ T	+id*id\$	reduce by $E \rightarrow T$
\$E	+id*id\$	shift
\$E+	id*id\$	shift
\$E+ id	*id\$	reduce by $F \rightarrow id$
\$E+ F	*id\$	reduce by $T \rightarrow F$
\$E+T	*id\$	shift
\$E+T*	id\$	shift
\$E+T* id	\$	reduce by $F \rightarrow id$
\$E+ T* F	\$	reduce by $T \rightarrow T*F$
\$ E+T	\$	reduce by $E \rightarrow E+T$
\$E	\$	accept

Parse Tree



Viable Prefixes

- The set of prefixes of right sentential forms that can appear on the stack of a shift-reduce parser are called *viable prefixes*.
- The stack contents must be a prefix of a right-sentential form.
- If the stack holds α and the rest of the input is x then a sequence of reductions will take αx to S .
- Not all prefixes of right-sentential forms can appear on the stack and the parser must not shift past the handle.

$$\text{Ex. } E \xRightarrow[\text{rm}]{*} F * \text{id} \xRightarrow[\text{rm}]{} (E) * \text{id}$$

- At various times the stack will hold $($, $(E$, and (E) but it must not hold $(E)^*$, since (E) is a handle which the parser must reduce to F before shifting $*$.

Operator-Precedence Parser

- **Operator grammar**
 - small, but an important class of grammars
 - we may have an efficient operator precedence parser (a shift-reduce parser) for an operator grammar.
- In an *operator grammar*, no production rule can have:
 - ϵ at the right side
 - two adjacent non-terminals at the right side.

- Ex:

$E \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

not operator grammar

$E \rightarrow EOE$

$E \rightarrow id$

$O \rightarrow + | * | /$

not operator grammar

$E \rightarrow E + E \mid$

$E * E \mid$

$E / E \mid id$

operator grammar

Precedence Relations

- In operator-precedence parsing, we define three disjoint precedence relations between certain pairs of terminals.

$a < \cdot b$ b has higher precedence than a

$a = \cdot b$ b has same precedence as a

$a \cdot > b$ b has lower precedence than a

- The determination of correct precedence relations between terminals are based on the traditional notions of associativity and precedence of operators. (Unary minus causes a problem).

Using Operator-Precedence Relations

- The intention of the precedence relations is to find the handle of a right-sentential form,
 - $\leq \cdot$ with marking the left end,
 - $= \cdot$ appearing in the interior of the handle, and
 - $\cdot >$ marking the right hand.
- In our input string $a_1 a_2 \dots a_n$, we insert the precedence relation between the pairs of terminals (the precedence relation holds between the terminals in that pair).

Using Operator -Precedence Relations

$E \rightarrow E+E \mid E-E \mid E * E \mid E/E \mid E^E \mid (E) \mid -E \mid id$

The partial operator-precedence table for this grammar

	id	+	*	\$
id		$\cdot >$	$\cdot >$	$\cdot >$
+	$< \cdot$	$\cdot >$	$< \cdot$	$\cdot >$
*	$< \cdot$	$\cdot >$	$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	

- Then the input string $id+id*id$ with the precedence relations inserted will be:

$\$ < \cdot id \cdot > + < \cdot id \cdot > * < \cdot id \cdot > \$$

To Find The Handles

1. Scan the string from left end until the first $\cdot >$ is encountered.
2. Then scan backwards (to the left) over any $=\cdot$ until a $<\cdot$ is encountered.
3. The handle contains everything to left of the first $\cdot >$ and to the right of the $<\cdot$ is encountered.

$\$ <\cdot \text{ id } \cdot > + <\cdot \text{ id } \cdot > * <\cdot \text{ id } \cdot > \$$

$E \rightarrow \text{id}$

$\$ \text{ id } + \text{id } * \text{id } \$$

$\$ <\cdot + <\cdot \text{ id } \cdot > * <\cdot \text{ id } \cdot > \$$

$E \rightarrow \text{id}$

$\$ E + \text{id } * \text{id } \$$

$\$ <\cdot + <\cdot * <\cdot \text{ id } \cdot > \$$

$E \rightarrow \text{id}$

$\$ E + E * \text{id } \$$

$\$ <\cdot + <\cdot * \cdot > \$$

$E \rightarrow E * E$

$\$ E + E * \cdot E \$$

$\$ <\cdot + \cdot > \$$

$E \rightarrow E + E$

$\$ E + E \$$

$\$ \$$

$\$ E \$$

Operator-Precedence Parsing Algorithm

- The input string is $w\$$, the initial stack is $\$$ and a table holds precedence relations between certain terminals

Algorithm:

set p to point to the first symbol of $w\$$;

repeat forever

if ($\$$ is on top of the stack **and** p points to $\$$) **then return**

else {

let a be the topmost terminal symbol on the stack and let b be the symbol pointed to by p ;

if ($a < \cdot b$ or $a = \cdot b$) **then {** */* SHIFT */*

push b onto the stack;

advance p to the next input symbol;

}

else if ($a > \cdot b$) **then** */* REDUCE */*

repeat pop stack

until (the top of stack terminal is related by $< \cdot$ to the terminal most recently popped);

else error();

}

Operator-Precedence Parsing Algorithm -- Example

<u>stack</u>	<u>input</u>	<u>action</u>
\$	id+id*id\$	$\$ < \cdot \text{id}$ shift
\$id	+id*id\$	$\text{id} \cdot > +$ reduce $E \rightarrow \text{id}$
\$	+id*id\$	shift
\$+	id*id\$	shift
\$+id	*id\$	$\text{id} \cdot > *$ reduce $E \rightarrow \text{id}$
\$+	*id\$	shift
\$+*	id\$	shift
\$+*id	\$	$\text{id} \cdot > \$$ reduce $E \rightarrow \text{id}$
\$+*	\$	$* \cdot > \$$ reduce $E \rightarrow E * E$
\$+	\$	$+ \cdot > \$$ reduce $E \rightarrow E + E$
\$	\$	accept

How to Create Operator-Precedence Relations

- We use associativity and precedence relations among operators.
- If operator O_1 has higher precedence than operator O_2 ,
 $\rightarrow O_1 \cdot > O_2$ and $O_2 < \cdot O_1$
- If operator O_1 and operator O_2 have equal precedence,
they are left-associative $\rightarrow O_1 \cdot > O_2$ and $O_2 \cdot > O_1$
they are right-associative $\rightarrow O_1 < \cdot O_2$ and $O_2 < \cdot O_1$
- For all operators O ,
 $O < \cdot \text{id}$, $\text{id} \cdot > O$, $O < \cdot ($, $(< \cdot O$, $O \cdot >)$, $) \cdot > O$, $O \cdot > \$$, and $\$ < \cdot O$
- Also, let
$$\begin{aligned}
 (= \cdot) \quad & \$ < \cdot (\quad \text{id} \cdot >) \quad) \cdot > \$ \\
 (< \cdot (\quad & \$ < \cdot \text{id} \quad \text{id} \cdot > \$ \quad) \cdot >) \\
 (< \cdot \text{id}
 \end{aligned}$$

Operator-Precedence Relations

	+	-	*	/	^	id	(	)	\$
+	$\cdot >$	$\cdot >$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
-	$\cdot >$	$\cdot >$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
*	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
/	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
^	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
id	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$			$\cdot >$	$\cdot >$
(	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$= \cdot$	
)	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$			$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$		

Handling Unary Minus

- Operator-Precedence parsing cannot handle the unary minus when we also the binary minus in our grammar.
- The best approach to solve this problem, let the lexical analyzer handle this problem.
 - The lexical analyzer will return two different operators for the unary minus and the binary minus.
 - The lexical analyzer will need a lookahead to distinguish the binary minus from the unary minus.
- Then, we make
 - $O \prec \text{unary-minus}$ for any operator
 - $\text{unary-minus} \succ O$ if unary-minus has higher precedence than O
 - $\text{unary-minus} \preceq O$ if unary-minus has lower (or equal) precedence than O

Operator-Precedence Grammars

Let G be an ϵ -free operator grammar (No ϵ -Production). For each terminal symbols a and b , the following conditions are satisfied.

1. $a \doteq b$, if $\exists$ a production in RHS of the form $\alpha a \beta b \gamma$, where β is either ϵ or a single non-terminal. Ex $S \rightarrow i C t S e S$ implies $i \doteq t$ and $t \doteq e$.
2. $a < \cdot b$ if for some non-terminal $A \exists$ a production in RHS of the form $A \rightarrow \alpha a A \beta$, and $A \Rightarrow^+ \gamma b \delta$ where γ is either ϵ or a single non-terminal. Ex $S \rightarrow i C t S$ and $C \Rightarrow^+ b$ implies $i < \cdot b$.
3. $a \cdot > b$ if for some non-terminal $A \exists$ a production in RHS of the form $A \rightarrow \alpha A b \beta$, and $A \Rightarrow^+ \gamma a \delta$ where δ is either ϵ or a single non-terminal. Ex $S \rightarrow i C t S$ and $C \Rightarrow^+ b$ implies $b \cdot > t$.

Example: $E \rightarrow E + E \mid E * E \mid (E) \mid id$ is not a Operator precedence Grammar

By Rule no. 3 we have $+ < \cdot +$ & $+ \cdot > +$. Whereas we can modify the Grammar as follow

$E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$

Operator Precedence Relations.

To find the Table we have to find the last & first terminal for each non-terminal as follows:

<u>Non terminal</u>	<u>First terminal</u>	<u>Last terminal</u>
E	$*, +, (, \mathbf{id}$	$*, +,), \mathbf{id}$
T	$*, (, \mathbf{id}$	$*,), \mathbf{id}$
F	$(, \mathbf{id}$	$), \mathbf{id}$

	+	*	(	)	id	\$
+	.>	<.	<.	.>	<.	.>
*	.>	.>	<.	.>	<.	.>
(	<.	<.	<.	$\doteq$	<.	
)	.>	.>		.>		.>
id	.>	.>		.>		.>
\$	<.	<.	<.		<.	

By Applying the Rule of Operator Precedence Grammar

Operator Precedence Relations, Continue....

To produce the Table we have to follow the procedure as:

$\text{LEADING}(A) = \{ a \mid A \Rightarrow^+ \gamma a \delta, \text{ where } \gamma \text{ is } \epsilon \text{ or a single non-terminal.} \}$

$\text{TRAILING}(A) = \{ a \mid A \Rightarrow^+ \gamma a \delta, \text{ where } \delta \text{ is } \epsilon \text{ or a single non-terminal.} \}$

Precedence Functions

- Compilers using operator precedence parsers do not need to store the table of precedence relations.
- The table can be encoded by two precedence functions f and g that map terminal symbols to integers.
- For symbols a and b .
 - $f(a) < g(b)$ whenever $a < \cdot b$
 - $f(a) = g(b)$ whenever $a = \cdot b$
 - $f(a) > g(b)$ whenever $a \cdot > b$

Algorithm for Constructing Precedence Functions

1. Create functions f_a and g_a for each terminal a and $\$$.
2. Partition the symbols in groups so that f_a and g_b are in the same group if $a =\bullet b$ (there can be symbols in the same group even if they are not connected by this relation);
3. Create a directed graph whose nodes are in the groups found in step (2). For each symbols a and b , place an edge from the group of g_b to the group of f_a if $a <\bullet b$, otherwise if $a \bullet > b$ place an edge from the group of f_a to that of g_b ;

Algorithm for Constructing Precedence Functions

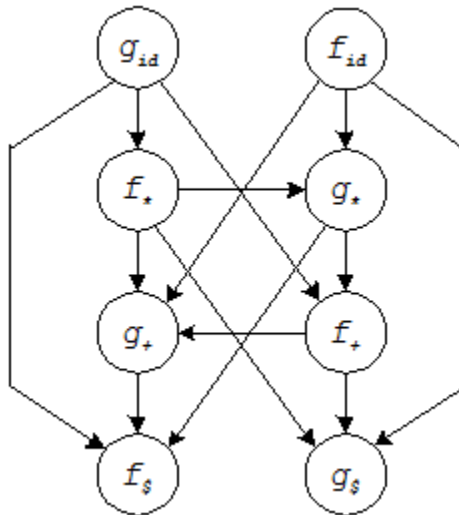
4. If the constructed graph has a cycle then no precedence functions exist. When there are no cycles then $f(a)$ and $f(b)$ are the lengths of the longest paths from the groups of f_a and g_b respectively.

Precedence Functions

Example : Precedence Table

	id	+	*	\$
id		$\cdot >$	$\cdot >$	$\cdot >$
+	$< \cdot$	$\cdot >$	$< \cdot$	$\cdot >$
*	$< \cdot$	$\cdot >$	$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$

Graph



Precedence Functions

	id	+	*	\$
f	4	2	4	0
g	5	1	3	0

Disadvantages of Operator Precedence Parsing

- **Disadvantages:**
 - It cannot handle the unary minus (the lexical analyzer should handle the unary minus).
 - Small class of grammars.
 - Difficult to decide which language is recognized by the grammar.
- **Advantages:**
 - simple
 - powerful enough for expressions in programming languages

Error Recovery in Operator-Precedence Parsing

Error Cases:

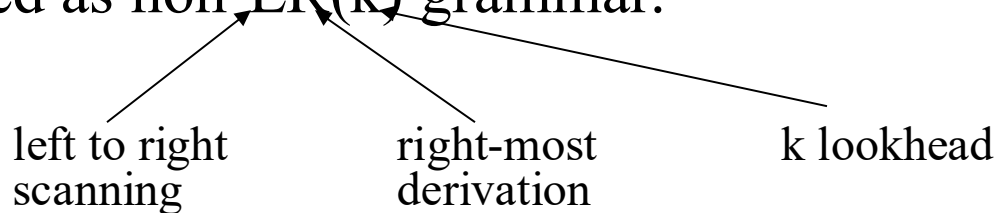
1. No relation holds between the terminal on the top of stack and the next input symbol.
2. A handle is found (reduction step), but there is no production with this handle as a right side

Error Recovery:

1. Each empty entry is filled with a pointer to an error routine.
2. Decides the popped handle “looks like” which right hand side. And tries to recover from that situation.

Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsers cannot be used.
- Stack contents and the next input symbol may not decide action:
 - **shift/reduce conflict**: Whether make a shift operation or a reduction.
 - **reduce/reduce conflict**: The parser cannot decide which of several reductions to make.
- If a shift-reduce parser cannot be used for a grammar, that grammar is called as non-LR(k) grammar.



- An ambiguous grammar can never be a LR grammar.

Shift-Reduce Parsers

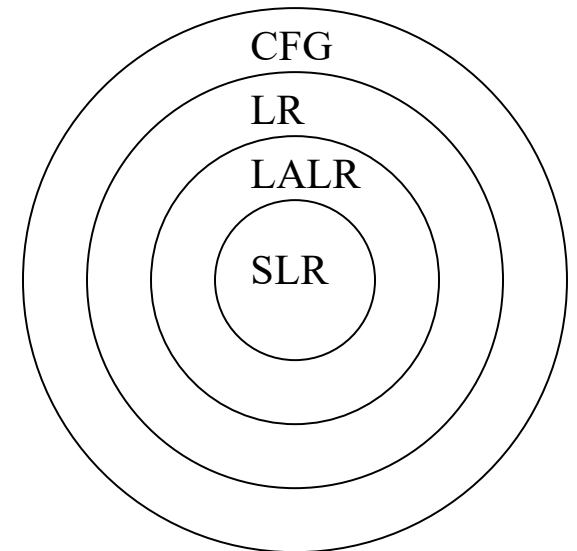
- There are two main categories of shift-reduce parsers

1. Operator-Precedence Parser

- simple, but only a small class of grammars.

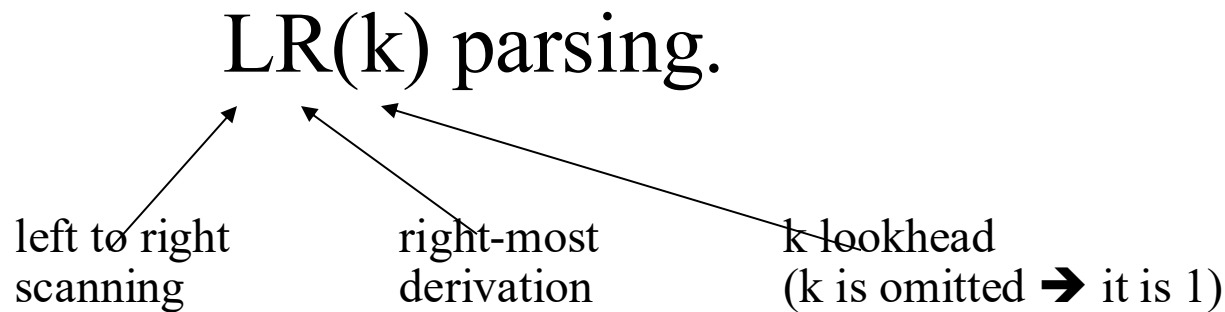
2. LR-Parsers

- covers wide range of grammars.
 - SLR – simple LR parser
 - LR – most general LR parser
 - LALR – intermediate LR parser (lookahead LR parser)
- SLR, LR and LALR work same, only their parsing tables are different.



LR Parsers

- The most powerful shift-reduce parsing (yet efficient) is:

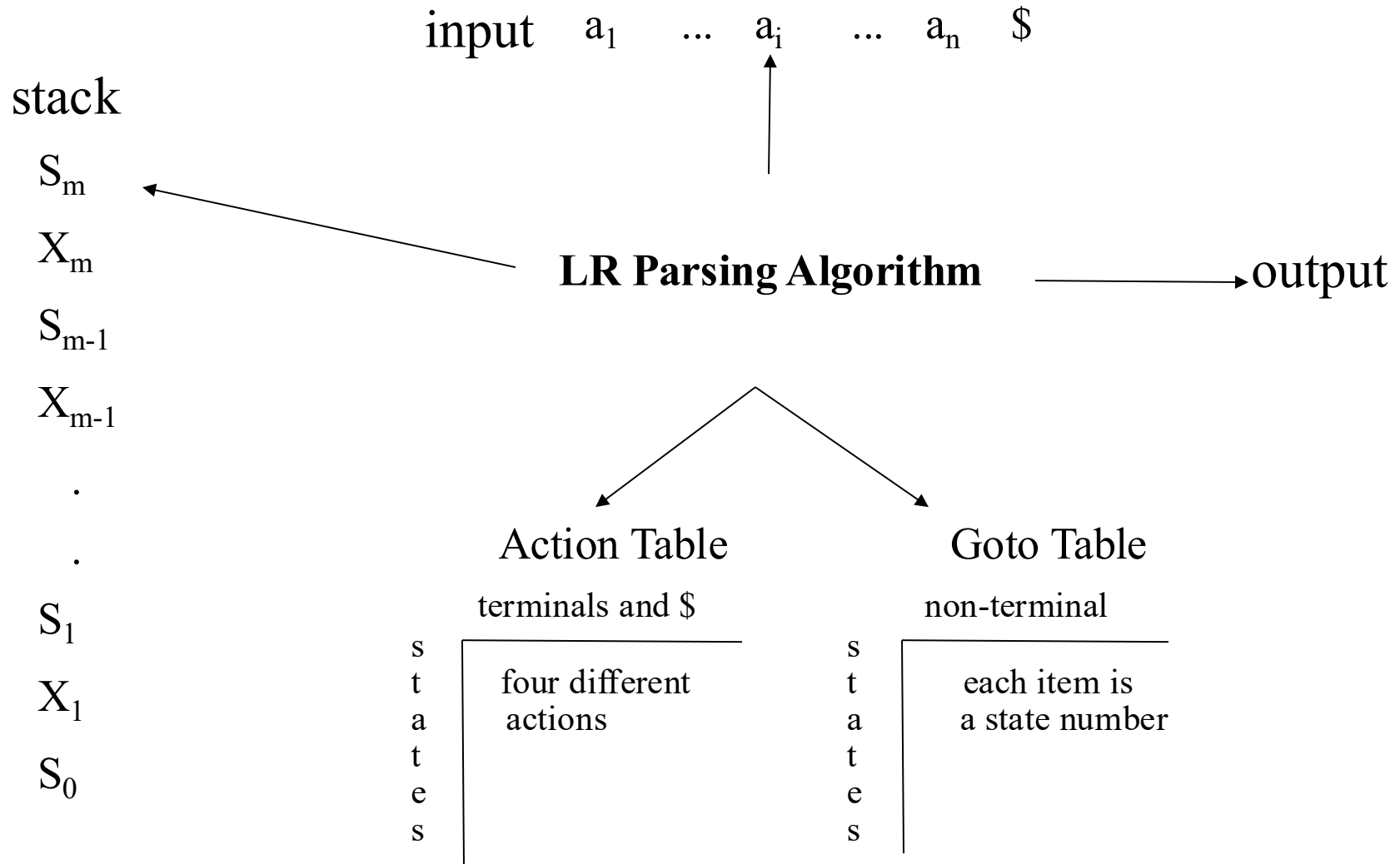


- LR parsing is attractive because:
 - LR parsing is most general non-backtracking shift-reduce parsing, yet it is still efficient.
 - The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.
$$\text{LL(1)-Grammars} \subset \text{LR(1)-Grammars}$$
 - An LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the input.

LR Parsers

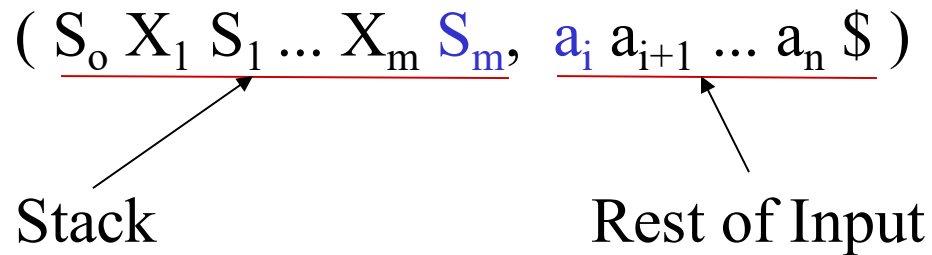
- **LR-Parsers**
 - covers wide range of grammars.
 - SLR – simple LR parser
 - LR – most general LR parser
 - LALR – intermediate LR parser (look-head LR parser)
 - SLR, LR and LALR work same (they used the same algorithm), only their parsing tables are different.

LR Parsing Algorithm



A Configuration of LR Parsing Algorithm

- A configuration of a LR parsing is:



- S_m and a_i decides the parser action by consulting the parsing action table. (*Initial Stack* contains just S_o)
- A configuration of a LR parsing represents the right sentential form:

$$X_1 \ \dots \ X_m \ a_i \ a_{i+1} \ \dots \ a_n \ \$$$

Actions of A LR-Parser

1. **shift s** -- shifts the next input symbol and the state **s** onto the stack

$(S_o X_1 S_1 \dots X_m S_m, a_i a_{i+1} \dots a_n \$) \rightarrow (S_o X_1 S_1 \dots X_m S_m \mathbf{a_i s}, a_{i+1} \dots a_n \$)$

1. **reduce $A \rightarrow \beta$** (or **rn** where n is a production number)

- pop $2|\beta|$ ($=r$) items from the stack;
- then push **A** and **s** where **s=goto[s_{m-r},A]**

$(S_o X_1 S_1 \dots X_m S_m, a_i a_{i+1} \dots a_n \$) \rightarrow (S_o X_1 S_1 \dots X_{m-r} \mathbf{S_{m-r} A s}, a_i \dots a_n \$)$

- Output is the reducing production reduce $A \rightarrow \beta$

1. **Accept** – Parsing successfully completed

1. **Error** -- Parser detected an error (an empty entry in the action table)

Reduce Action

- pop $2|\beta|$ ($=r$) items from the stack; let us assume that $\beta = Y_1 Y_2 \dots Y_r$
- then push A and s where $s = \text{goto}[s_{m-r}, A]$

$$\begin{aligned}
 & (S_o \ X_1 \ S_1 \ \dots \ X_{m-r} \ S_{m-r} \ Y_1 \ S_{m-r} \ \dots \ Y_r \ S_m, a_i \ a_{i+1} \ \dots \ a_n \ \$) \\
 & \quad \rightarrow (S_o \ X_1 \ S_1 \ \dots \ X_{m-r} \ S_{m-r} \ A \ s, a_i \ \dots \ a_n \ \$)
 \end{aligned}$$

- In fact, $Y_1 Y_2 \dots Y_r$ is a handle.

$$X_1 \ \dots \ X_{m-r} \ A \ a_i \ \dots \ a_n \ \$ \Rightarrow X_1 \ \dots \ X_m \ Y_1 \dots Y_r \ a_i \ a_{i+1} \ \dots \ a_n \ \$$$

(SLR) Parsing Tables for Expression Grammar

- 1) $E \rightarrow E+T$
- 2) $E \rightarrow T$
- 3) $T \rightarrow T*F$
- 4) $T \rightarrow F$
- 5) $F \rightarrow (E)$
- 6) $F \rightarrow id$

Action Table							Goto Table			
state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

Actions of A (S)LR-Parser -- Example

<u>stack</u>	<u>input</u>	<u>action</u>	<u>output</u>
0	id*id+id\$	shift 5	
0id5	*id+id\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0F3	*id+id\$	reduce by $T \rightarrow F$	$T \rightarrow F$
0T2	*id+id\$	shift 7	
0T2*7	id+id\$	shift 5	
0T2*7id5	+id\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0T2*7F10	+id\$	reduce by $T \rightarrow T * F$	$T \rightarrow T * F$
0T2	+id\$	reduce by $E \rightarrow T$	$E \rightarrow T$
0E1	+id\$	shift 6	
0E1+6	id\$	shift 5	
0E1+6id5	\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0E1+6F3	\$	reduce by $T \rightarrow F$	$T \rightarrow F$
0E1+6T9	\$	reduce by $E \rightarrow E + T$	$E \rightarrow E + T$
0E1	\$	accept	

Constructing SLR Parsing Tables – LR(0) Item

- An **LR(0) item** of a grammar G is a production of G a dot at the some position of the right side.
- Ex: $A \rightarrow aBb$ Possible LR(0) Items: $A \rightarrow \bullet aBb$
 (four different possibility) $A \rightarrow a \bullet Bb$
 $A \rightarrow aB \bullet b$
 $A \rightarrow aBb \bullet$
- Sets of LR(0) items will be the states of action and goto table of the SLR parser.
- A collection of sets of LR(0) items (**the canonical LR(0) collection**) is the basis for constructing SLR parsers.
- Augmented Grammar:*
 G' is G with a new production rule $S' \rightarrow S$ where S' is the new starting symbol.

The Closure Operation

- If I is a set of LR(0) items for a grammar G , then ***closure(I)*** is the set of LR(0) items constructed from I by the two rules:
 1. Initially, every LR(0) item in I is added to $\text{closure}(I)$.
 2. If $A \rightarrow \alpha \bullet B\beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production rule of G ; then $B \rightarrow \bullet \gamma$ will be in the $\text{closure}(I)$.
We will apply this rule until no more new LR(0) items can be added to $\text{closure}(I)$.

The Closure Operation -- Example

$E' \rightarrow E$

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T*F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{id}$

$\text{closure}(\{E' \rightarrow \bullet E\}) =$

$\{ E' \rightarrow \bullet E \}$ ← kernel items

$E \rightarrow \bullet E+T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T*F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet \text{id} \}$

Goto Operation

- If I is a set of LR(0) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is defined as follows:
 - If $A \rightarrow \alpha \bullet X \beta$ in I
then every item in $\text{closure}(\{A \rightarrow \alpha X \bullet \beta\})$ will be in $\text{goto}(I, X)$.

Example:

$I = \{ \begin{array}{l} E' \rightarrow \bullet E, \quad E \rightarrow \bullet E + T, \quad E \rightarrow \bullet T, \\ T \rightarrow \bullet T * F, \quad T \rightarrow \bullet F, \\ F \rightarrow \bullet (E), \quad F \rightarrow \bullet \text{id} \end{array} \}$

$\text{goto}(I, E) = \{ E' \rightarrow E \bullet, E \rightarrow E \bullet + T \}$

$\text{goto}(I, T) = \{ E \rightarrow T \bullet, T \rightarrow T \bullet * F \}$

$\text{goto}(I, F) = \{ T \rightarrow F \bullet \}$

$\text{goto}(I, () = \{ F \rightarrow (\bullet E), E \rightarrow \bullet E + T, E \rightarrow \bullet T, T \rightarrow \bullet T * F, T \rightarrow \bullet F, \\ F \rightarrow \bullet (E), F \rightarrow \bullet \text{id} \}$

$\text{goto}(I, \text{id}) = \{ F \rightarrow \text{id} \bullet \}$

Construction of The Canonical LR(0) Collection

- To create the SLR parsing tables for a grammar G , we will create the canonical LR(0) collection of the grammar G' .
- *Algorithm:*
 - C is $\{ \text{closure}(\{S' \rightarrow \bullet S\}) \}$
 - repeat** the followings until no more set of LR(0) items can be added to C .
 - for each** I in C and each grammar symbol X
 - if** $\text{goto}(I, X)$ is not empty and not in C
 - add $\text{goto}(I, X)$ to C
- goto function is a DFA on the sets in C .

The Canonical LR(0) Collection -- Example

$I_0: E' \rightarrow .E$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_1: E' \rightarrow E.I_6: E \rightarrow E+.T$

$E \rightarrow E+.T$

$I_2: E \rightarrow T.$

$T \rightarrow T.*F$

$I_3: T \rightarrow F.$

$I_4: F \rightarrow (E)$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_5: F \rightarrow id.$

$I_9: E \rightarrow E+T.$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_7: T \rightarrow T*.F$

$F \rightarrow .(E)$

$F \rightarrow .id$

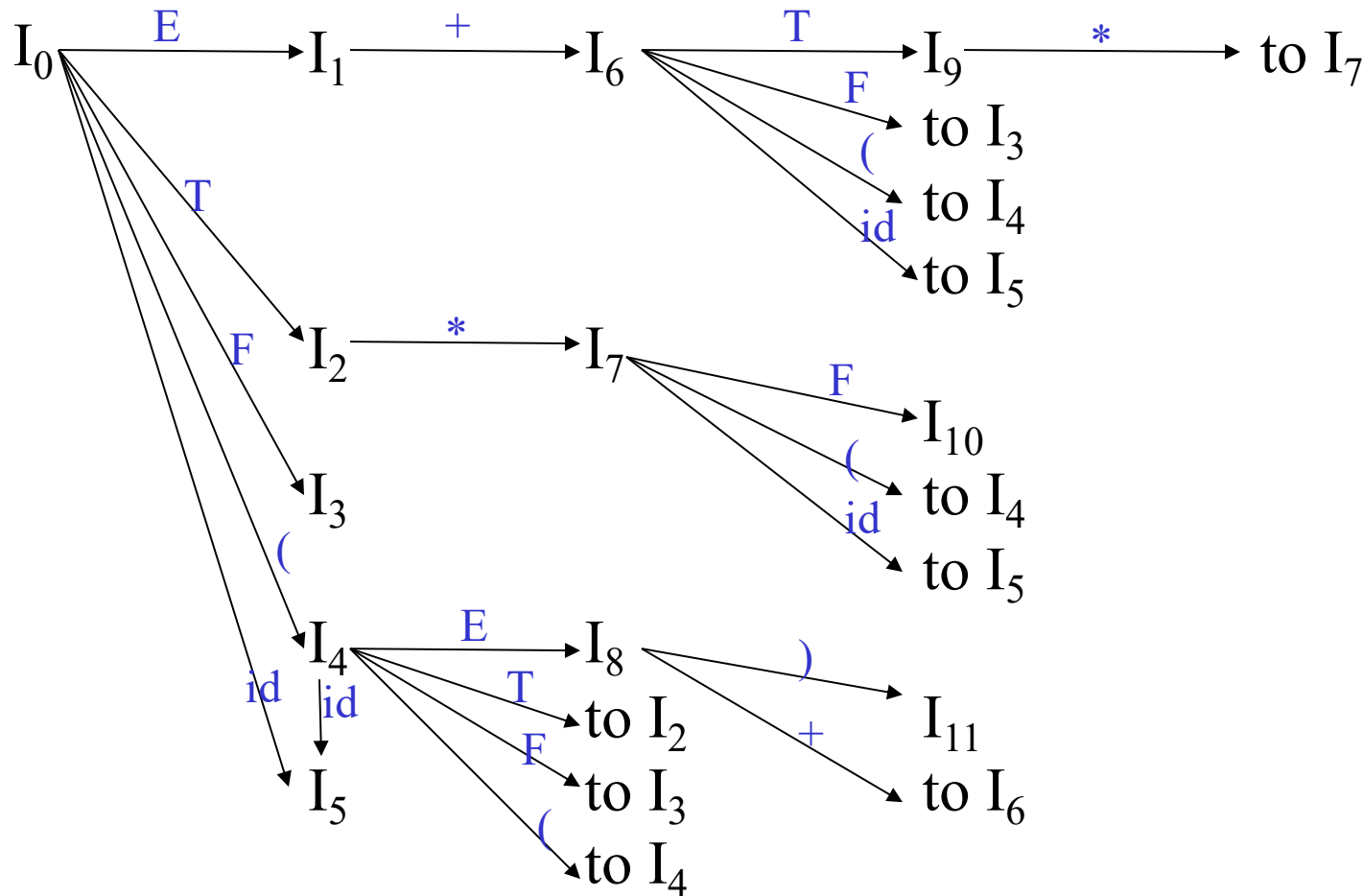
$I_8: F \rightarrow (E.)$

$E \rightarrow E.+T$

$I_{10}: T \rightarrow T*F.$

$I_{11}: F \rightarrow (E).$

Transition Diagram (DFA) of Goto Function



Constructing SLR Parsing Table

(of an augmented grammar G')

1. Construct the canonical collection of sets of LR(0) items for G' .
 $C \leftarrow \{I_0, \dots, I_n\}$
1. Create the parsing action table as follows
 - If a is a terminal, $A \rightarrow \alpha.a\beta$ in I_i and $\text{goto}(I_i, a) = I_j$ then $\text{action}[i, a]$ is *shift j*.
 - If $A \rightarrow \alpha.$ is in I_i , then $\text{action}[i, a]$ is *reduce $A \rightarrow \alpha$* for all a in $\text{FOLLOW}(A)$ where $A \neq S'$.
 - If $S' \rightarrow S.$ is in I_i , then $\text{action}[i, \$]$ is *accept*.
 - If any conflicting actions generated by these rules, the grammar is not SLR(1).
1. Create the parsing goto table
 - for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$
1. All entries not defined by (2) and (3) are errors.
1. Initial state of the parser contains $S' \rightarrow .S$

Parsing Tables of Expression Grammar

Action Table

Goto Table

state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

SLR(1) Grammar

- An LR parser using SLR(1) parsing tables for a grammar G is called as the SLR(1) parser for G .
- If a grammar G has an SLR(1) parsing table, it is called SLR(1) grammar (or SLR grammar in short).
- Every SLR grammar is unambiguous, but every unambiguous grammar is not a SLR grammar.

shift/reduce and reduce/reduce conflicts

- If a state does not know whether it will make a shift operation or reduction for a terminal, we say that there is a **shift/reduce conflict**.
- If a state does not know whether it will make a reduction operation using the production rule i or j for a terminal, we say that there is a **reduce/reduce conflict**.
- If the SLR parsing table of a grammar G has a conflict, we say that that grammar is not SLR grammar.

Conflict Example

$S \rightarrow L=R$

$S \rightarrow R$

$L \rightarrow *R$

$L \rightarrow \text{id}$

$R \rightarrow L$

$I_0: S' \rightarrow .S$

$S \rightarrow .L=R$

$S \rightarrow .R$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$R \rightarrow .L$

$I_1: S' \rightarrow S.$

$I_2: S \rightarrow L.=R$

$R \rightarrow L.$

$I_3: S \rightarrow R.$

$I_4: L \rightarrow *.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$I_5: L \rightarrow \text{id}.$

$I_6: S \rightarrow L=.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$I_7: L \rightarrow *.R.$

$I_8: R \rightarrow L.$

$I_9: S \rightarrow L=R.$

Problem

$\text{FOLLOW}(R) = \{=, \$\}$

= $\swarrow$ shift 6

$\searrow$ reduce by $R \rightarrow L$

shift/reduce conflict

Conflict Example2

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

$I_0: S' \rightarrow .S$

$S \rightarrow .AaAb$

$S \rightarrow .BbBa$

$A \rightarrow .$

$B \rightarrow .$

Problem

$\text{FOLLOW}(A) = \{a, b\}$

$\text{FOLLOW}(B) = \{a, b\}$

a $\begin{cases} \rightarrow \text{reduce by } A \rightarrow \epsilon \\ \rightarrow \text{reduce by } B \rightarrow \epsilon \end{cases}$

reduce/reduce conflict

b $\begin{cases} \rightarrow \text{reduce by } A \rightarrow \epsilon \\ \rightarrow \text{reduce by } B \rightarrow \epsilon \end{cases}$

reduce/reduce conflict

Constructing Canonical LR(1) Parsing Tables

- In SLR method, the state i makes a reduction by $A \rightarrow \alpha$ when the current token is a :
 - if the $A \rightarrow \alpha \cdot$ in the I_i and a is $\text{FOLLOW}(A)$
- In some situations, βA cannot be followed by the terminal a in a right-sentential form when $\beta \alpha$ and the state i are on the top stack. This means that making reduction in this case is not correct.

$S \rightarrow AaAb$

$S \Rightarrow AaAb \Rightarrow Aab \Rightarrow ab$

$S \Rightarrow BbBa \Rightarrow Bba \Rightarrow ba$

$S \rightarrow BbBa$

$A \rightarrow \epsilon$

$Aab \Rightarrow \epsilon ab$

$Bba \Rightarrow \epsilon ba$

$B \rightarrow \epsilon$

$AaAb \Rightarrow Aa \epsilon b$

$BbBa \Rightarrow Bb \epsilon a$

LR(1) Item

- To avoid some of invalid reductions, the states need to carry more information.
- Extra information is put into a state by including a terminal symbol as a second component in an item.

- A LR(1) item is:

$$A \rightarrow \alpha \cdot \beta, a$$

where **a** is the look-head of the LR(1) item
(**a** is a terminal or end-marker.)

LR(1) Item (cont.)

- When β (in the LR(1) item $A \rightarrow \alpha.\beta,a$) is not empty, the look-head does not have any affect.
- When β is empty ($A \rightarrow \alpha.,a$), we do the reduction by $A \rightarrow \alpha$ only if the next input symbol is **a** (not for any terminal in FOLLOW(A)).
- A state will contain $A \rightarrow \alpha.,a_1$ where $\{a_1, \dots, a_n\} \subseteq \text{FOLLOW}(A)$

...

$A \rightarrow \alpha.,a_n$

Canonical Collection of Sets of LR(1) Items

- The construction of the canonical collection of the sets of LR(1) items are similar to the construction of the canonical collection of the sets of LR(0) items, except that *closure* and *goto* operations work a little bit different.

closure(I) is: (where I is a set of LR(1) items)

- every LR(1) item in I is in closure(I)
- if $A \rightarrow \alpha \cdot B \beta, a$ in closure(I) and $B \rightarrow \gamma$ is a production rule of G; then $B \rightarrow \cdot \gamma, b$ will be in the closure(I) for each terminal b in FIRST(βa) .

goto operation

- If I is a set of LR(1) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is defined as follows:
 - If $A \rightarrow \alpha.X\beta, a$ in I
then every item in $\text{closure}(\{A \rightarrow \alpha.X.\beta, a\})$ will be in $\text{goto}(I, X)$.

Construction of The Canonical LR(1) Collection

- *Algorithm:*

C is $\{ \text{closure}(\{S' \rightarrow .S, \$\}) \}$

repeat the followings until no more set of LR(1) items can be added to C .

for each I in C and each grammar symbol X

if $\text{goto}(I, X)$ is not empty and not in C

 add $\text{goto}(I, X)$ to C

- goto function is a DFA on the sets in C .

A Short Notation for The Sets of LR(1) Items

- A set of LR(1) items containing the following items

$$A \rightarrow \alpha \cdot \beta, a_1$$

...

$$A \rightarrow \alpha \cdot \beta, a_n$$

can be written as

$$A \rightarrow \alpha \cdot \beta, a_1/a_2/.../a_n$$

Canonical LR(1) Collection -- Example

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

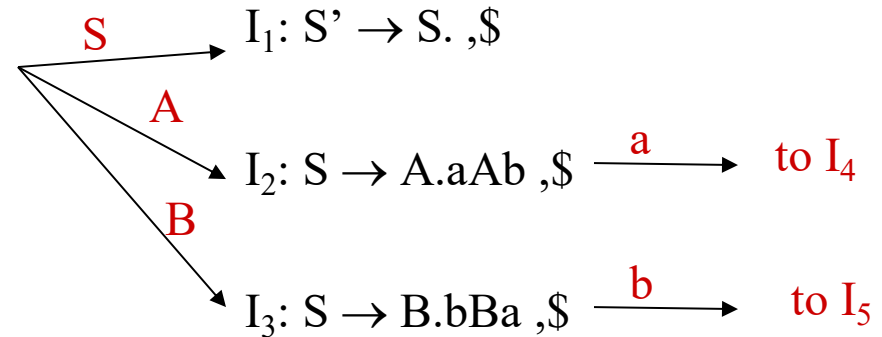
$I_0: S' \rightarrow .S, \$$

$S \rightarrow .AaAb, \$$

$S \rightarrow .BbBa, \$$

$A \rightarrow ., a$

$B \rightarrow ., b$



$I_4: S \rightarrow Aa.Ab, \$ \xrightarrow{A} I_6: S \rightarrow AaA.b, \$ \xrightarrow{a} I_8: S \rightarrow AaAb., \$$
 $A \rightarrow ., b$

$I_5: S \rightarrow Bb.Ba, \$ \xrightarrow{B} I_7: S \rightarrow BbB.a, \$ \xrightarrow{b} I_9: S \rightarrow BbBa., \$$
 $B \rightarrow ., a$

Canonical LR(1) Collection – Example2

$S' \rightarrow S$

$I_0: S' \rightarrow .S, \$$

1) $S \rightarrow L=R$

$S \rightarrow .L=R, \$$

2) $S \rightarrow R$

$S \rightarrow .R, \$$

3) $L \rightarrow *R$

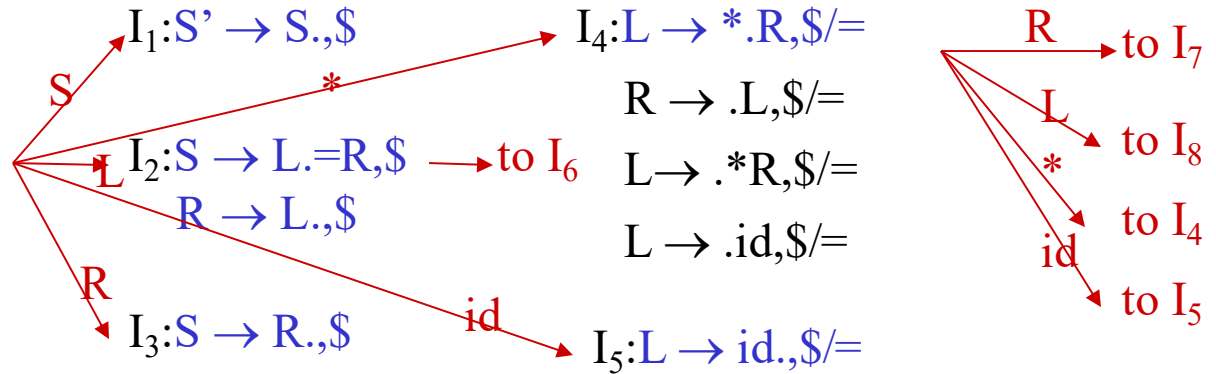
$L \rightarrow .*R, \$/=$

4) $L \rightarrow id$

$L \rightarrow .id, \$/=$

5) $R \rightarrow L$

$R \rightarrow .L, \$$

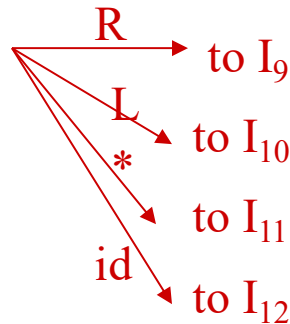


$I_6: S \rightarrow L=.R, \$$

$R \rightarrow .L, \$$

$L \rightarrow .*R, \$$

$L \rightarrow .id, \$$



$I_9: S \rightarrow L=R., \$$

$I_{10}: R \rightarrow L., \$$

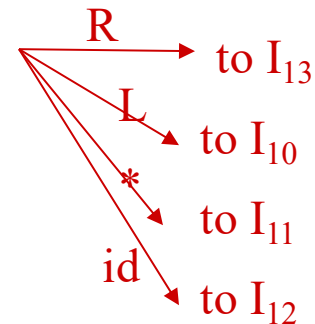
$I_{11}: L \rightarrow *.R, \$$

$R \rightarrow .L, \$$

$L \rightarrow .*R, \$$

$L \rightarrow .id, \$$

$I_{12}: L \rightarrow id., \$$



$I_{13}: L \rightarrow *.R., \$$

I_4 and I_{11}

I_5 and I_{12}

I_7 and I_{13}

I_8 and I_{10}

$I_7: L \rightarrow *.R., \$/=$

$I_8: R \rightarrow L., \$/=$

Construction of LR(1) Parsing Tables

1. Construct the canonical collection of sets of LR(1) items for G' .

$$C \leftarrow \{I_0, \dots, I_n\}$$

1. Create the parsing action table as follows
 - If a is a terminal, $A \rightarrow \alpha \cdot a \beta$, b in I_i and $\text{goto}(I_i, a) = I_j$ then $\text{action}[i, a]$ is *shift j*.
 - If $A \rightarrow \alpha \cdot$, a is in I_i , then $\text{action}[i, a]$ is *reduce $A \rightarrow \alpha$* where $A \neq S'$.
 - If $S' \rightarrow S \cdot, \$$ is in I_i , then $\text{action}[i, \$]$ is *accept*.
 - If any conflicting actions generated by these rules, the grammar is not LR(1).
1. Create the parsing goto table
 - for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$
1. All entries not defined by (2) and (3) are errors.
1. Initial state of the parser contains $S' \rightarrow \cdot S, \$$

LR(1) Parsing Tables – (for Example2)

	id	*	=	\$		S	L	R
0	s5	s4				1	2	3
1				acc				
2			s6	r5				
3				r2				
4	s5	s4					8	7
5			r4	r4				
6	s12	s11					10	9
7			r3	r3				
8			r5	r5				
9				r1				
10				r5				
11	s12	s11					10	13
12				r4				
13				r3				

no shift/reduce or
no reduce/reduce conflict



so, it is a LR(1) grammar

LALR Parsing Tables

- **LALR** stands for **LookAhead LR**.
- LALR parsers are often used in practice because LALR parsing tables are smaller than LR(1) parsing tables.
- The number of states in SLR and LALR parsing tables for a grammar G are equal.
- But LALR parsers recognize more grammars than SLR parsers.
- *yacc* creates a LALR parser for the given grammar.
- A state of LALR parser will be again a set of LR(1) items.

Creating LALR Parsing Tables

Canonical LR(1) Parser



LALR Parser

shrink # of states

- This shrink process may introduce a **reduce/reduce** conflict in the resulting LALR parser (so the grammar is NOT LALR)
- But, this shrink process does not produce a **shift/reduce** conflict.

The Core of A Set of LR(1) Items

- The core of a set of LR(1) items is the set of its first component.

Ex: $S \rightarrow L \bullet = R, \$$ $\rightarrow$ $S \rightarrow L \bullet = R$ $\leftarrow$ Core
 $R \rightarrow L \bullet, \$$ $R \rightarrow L \bullet$

- We will find the states (sets of LR(1) items) in a canonical LR(1) parser with same cores. Then we will merge them as a single state.

$I_1: L \rightarrow id \bullet, =$ A new state: $I_{12}: L \rightarrow id \bullet, =$
 $\rightarrow$ $L \rightarrow id \bullet, \$$
 $I_2: L \rightarrow id \bullet, \$$ have same core, merge them

- We will do this for all states of a canonical LR(1) parser to get the states of the LALR parser.
- In fact, the number of the states of the LALR parser for a grammar will be equal to the number of states of the SLR parser for that grammar.

Creation of LALR Parsing Tables

- Create the canonical LR(1) collection of the sets of LR(1) items for the given grammar.
- Find each core; find all sets having that same core; replace those sets having same cores with a single set which is their union.

$$C = \{I_0, \dots, I_n\} \rightarrow C' = \{J_1, \dots, J_m\} \quad \text{where } m \leq n$$

- Create the parsing tables (action and goto tables) same as the construction of the parsing tables of LR(1) parser.
 - Note that: If $J = I_1 \cup \dots \cup I_k$ since $I_1, \dots, I_k$ have same cores
 $\rightarrow$ cores of $\text{goto}(I_1, X), \dots, \text{goto}(I_k, X)$ must be same.
 - So, $\text{goto}(J, X) = K$ where K is the union of all sets of items having same cores as $\text{goto}(I_1, X)$.
- If no conflict is introduced, the grammar is LALR(1) grammar.
(We may only introduce reduce/reduce conflicts; we cannot introduce a shift/reduce conflict)

Shift/Reduce Conflict

- We say that we cannot introduce a shift/reduce conflict during the shrink process for the creation of the states of a LALR parser.
- Assume that we can introduce a shift/reduce conflict. In this case, a state of LALR parser must have:

$$A \rightarrow \alpha \bullet, a \quad \text{and} \quad B \rightarrow \beta \bullet a \gamma, b$$

- This means that a state of the canonical LR(1) parser must have:

$$A \rightarrow \alpha \bullet, a \quad \text{and} \quad B \rightarrow \beta \bullet a \gamma, c$$

But, this state has also a shift/reduce conflict. i.e. The original canonical LR(1) parser has a conflict.

(Reason for this, the shift operation does not depend on lookaheads)

Reduce/Reduce Conflict

- But, we may introduce a reduce/reduce conflict during the shrink process for the creation of the states of a LALR parser.

$I_1 : A \rightarrow \alpha \bullet, a$

$B \rightarrow \beta \bullet, b$

$I_2 : A \rightarrow \alpha \bullet, b$

$B \rightarrow \beta \bullet, c$

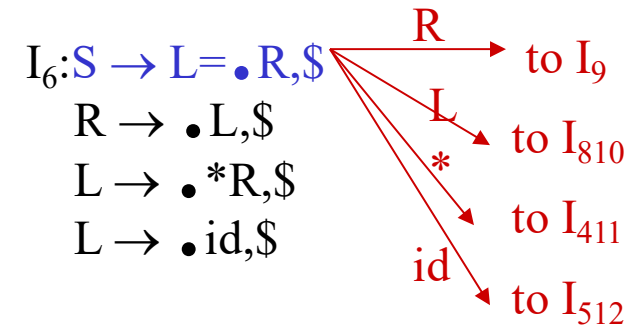
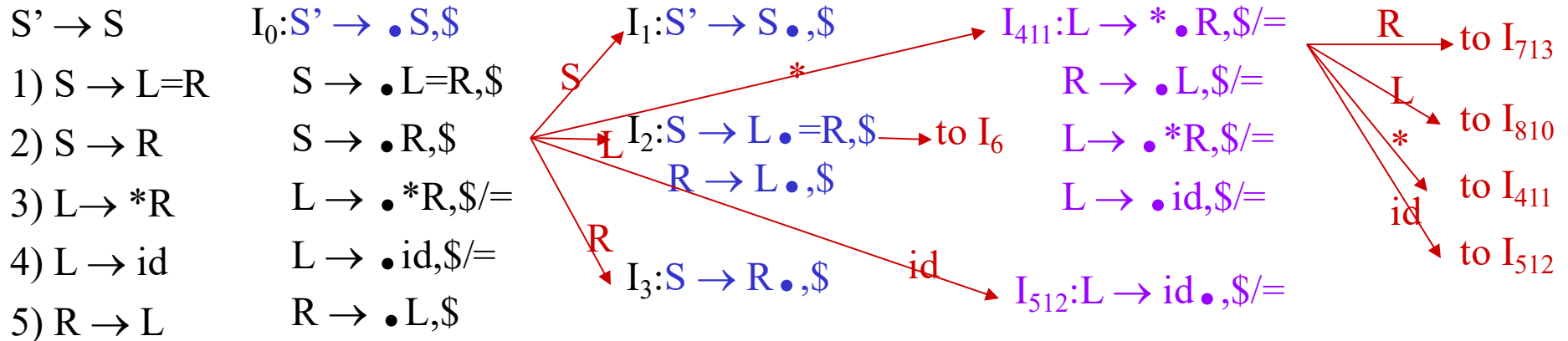


$I_{12} : A \rightarrow \alpha \bullet, a/b$

$B \rightarrow \beta \bullet, b/c$

➔ reduce/reduce conflict

Canonical LALR(1) Collection – Example2



$I_{713}: L \rightarrow *R \bullet, \$/=$

$I_{810}: R \rightarrow L \bullet, \$/=$

Same Cores
 I_4 and I_{11}

I_5 and I_{12}

I_7 and I_{13}

I_8 and I_{10}

LALR(1) Parsing Tables – (for Example2)

	id	*	=	\$	S	L	R
0	s5	s4			1	2	3
1				acc			
2			s6	r5			
3				r2			
4	s5	s4				8	7
5			r4	r4			
6	s12	s11				10	9
7			r3	r3			
8			r5	r5			
9				r1			

no shift/reduce or
no reduce/reduce conflict



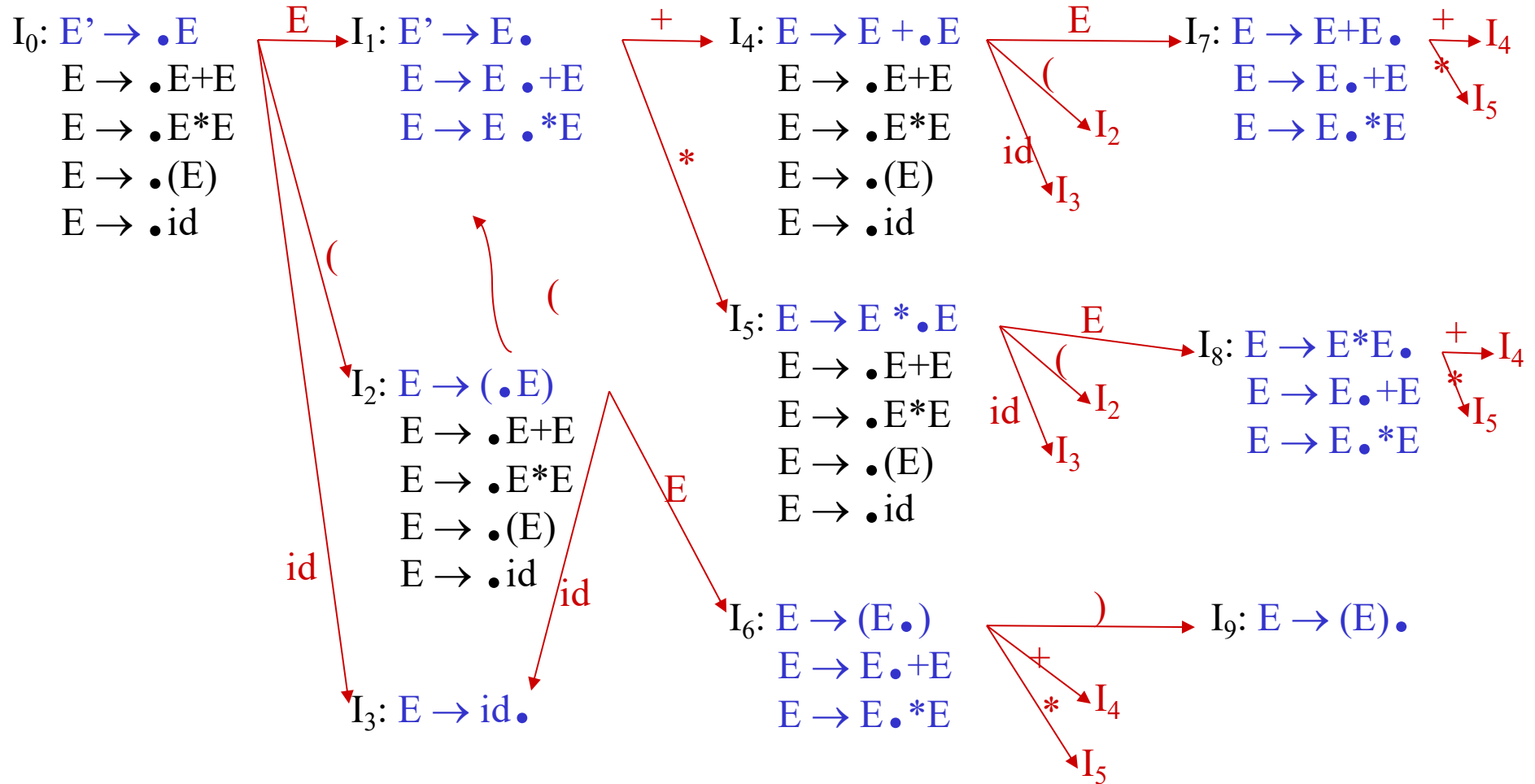
so, it is a LALR(1) grammar

Using Ambiguous Grammars

- All grammars used in the construction of LR-parsing tables must be un-ambiguous.
- Can we create LR-parsing tables for ambiguous grammars ?
 - Yes, but they will have conflicts.
 - We can resolve these conflicts in favor of one of them to disambiguate the grammar.
 - At the end, we will have again an unambiguous grammar.
- Why we want to use an ambiguous grammar?
 - Some of the ambiguous grammars are **much natural**, and a corresponding unambiguous grammar can be very complex.
 - Usage of an ambiguous grammar may **eliminate unnecessary reductions**.
- Ex.

$E \rightarrow E+E \mid E * E \mid (E) \mid \text{id}$	$\rightarrow$	$E \rightarrow E+T \mid T$
		$T \rightarrow T * F \mid F$
		$F \rightarrow (E) \mid \text{id}$

Sets of LR(0) Items for Ambiguous Grammar



SLR-Parsing Tables for Ambiguous Grammar

$$\text{FOLLOW}(E) = \{ \$, +, *,) \}$$

State I_7 has shift/reduce conflicts for symbols $+$ and $*$.

$$I_0 \xrightarrow{E} I_1 \xrightarrow{+} I_4 \xrightarrow{E} I_7$$

when current token is $+$

shift $\rightarrow$ $+$ is right-associative

reduce $\rightarrow$ $+$ is left-associative

when current token is $*$

shift $\rightarrow$ $*$ has higher precedence than $+$

reduce $\rightarrow$ $+$ has higher precedence than $*$

SLR-Parsing Tables for Ambiguous Grammar

$$\text{FOLLOW}(E) = \{ \$, +, *,) \}$$

State I_8 has shift/reduce conflicts for symbols $+$ and $*$.

$$I_0 \xrightarrow{E} I_1 \xrightarrow{*} I_5 \xrightarrow{E} I_7$$

when current token is $*$

shift $\rightarrow$ $*$ is right-associative

reduce $\rightarrow$ $*$ is left-associative

when current token is $+$

shift $\rightarrow$ $+$ has higher precedence than $*$

reduce $\rightarrow$ $*$ has higher precedence than $+$

SLR-Parsing Tables for Ambiguous Grammar

		Action					Goto	
	id	+	*	(	)	\$		E
0	s3			s2				1
1		s4	s5			acc		
2	s3			s2				6
3		r4	r4		r4	r4		
4	s3			s2				7
5	s3			s2				8
6		s4	s5		s9			
7		r1	s5		r1	r1		
8		r2	r2		r2	r2		
9		r3	r3		r3	r3		

Error Recovery in LR Parsing

- An LR parser will detect an error when it consults the parsing action table and finds an error entry. All empty entries in the action table are error entries.
- Errors are never detected by consulting the goto table.
- An LR parser will announce error as soon as there is no valid continuation for the scanned portion of the input.
- A canonical LR parser (LR(1) parser) will never make even a single reduction before announcing an error.
- The SLR and LALR parsers may make several reductions before announcing an error.
- But, all LR parsers (LR(1), LALR and SLR parsers) will never shift an erroneous input symbol onto the stack.

Panic Mode Error Recovery in LR Parsing

- Scan down the stack until a state **s** with a goto on a particular nonterminal **A** is found. (Get rid of everything from the stack before this state **s**).
- Discard zero or more input symbols until a symbol **a** is found that can legitimately follow **A**.
 - The symbol **a** is simply in FOLLOW(**A**), but this may not work for all situations.
- The parser stacks the nonterminal **A** and the state **goto[s,A]**, and it resumes the normal parsing.
- This nonterminal **A** is normally is a basic programming block (there can be more than one choice for **A**).
 - stmt, expr, block, ...

Phrase-Level Error Recovery in LR Parsing

- Each empty entry in the action table is marked with a specific error routine.
- An error routine reflects the error that the user most likely will make in that case.
- An error routine inserts the symbols into the stack or the input (or it deletes the symbols from the stack and the input, or it can do both insertion and deletion).
 - missing operand
 - unbalanced right parenthesis